```
package practice;

public class BookRunner {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        Book tamil = new Book();
        Book english = new Book();

        tamil.start();
        english.start();

        tamil.setNoOfCopies(100);
        System.out.println(tamil.getNoOfCopies());
        english.setNoOfCopies(10);
        System.out.println(english.getNoOfCopies());

        tamil.increaseNoOfCopies(100);

        english.increaseNoOfCopies(80);
        tamil.decreaseNoOfCopies(0);
        english.decreaseNoOfCopies(50);

        //System.out.println(noOfCopies);

    }

}

package practice;

public class Book {
private int noOfCopies;
public void setNoOfCopies(int noOfCopies) {
    if(noOfCopies > 0)
    this.noOfCopies = noOfCopies;
 }
public void increaseNoOfCopies(int howMany) {
    setNoOfCopies(this.noOfCopies + howMany);
```

```
}
public void decreaseNoOfCopies(int howMany) {
    setNoOfCopies(this.noOfCopies - howMany);
}
void start()
{
    System.out.println("Reading book");
}
public int getNoOfCopies() {
    return noOfCopies;
}



}
```

Explain me this program step by step also help me to print values of increase and decrease book separately

Here's what your program is doing, in the exact order it runs, and why you're not seeing the "increase/decrease" results printed yet.

What each class is for

`Book`

Represents a book with one piece of data:

- `private int noOfCopies;`
  This is the number of copies for that **one Book object**.

It has methods to:

- set copies safely (`setNoOfCopies`)
- increase copies (`increaseNoOfCopies`)
- decrease copies (`decreaseNoOfCopies`)
- show a message (`start`)
- read current value (`getNoOfCopies`)

`BookRunner`

This is the main program that creates Book objects and calls their methods.

Step-by-step execution (what happens in `main`)

1) Create two separate Book objects

Java

Now you have two **independent** objects:

- `tamil.noOfCopies` starts as `0` (default int value)
- `english.noOfCopies` starts as `0`

2) Call `start()` on both

Java

Each prints:

```
Reading book
Reading book
```

3) Set initial copies and print them

```java
tamil.setNoOfCopies(100);
System.out.println(tamil.getNoOfCopies());
```

Java

Important detail:

`setNoOfCopies(int noOfCopies)` only changes the value if `noOfCopies > 0`.

So after this:

- tamil = 100 → printed `100`
- english = 10 → printed `10`

4) Increase copies (but you didn't print after increasing)

Java

These methods update the value, but **they don't print anything**, so you won't see it unless you print.

5) Decrease copies

Java

No prints here either.

Why your `decreaseNoOfCopies` can behave "weird"

Because `setNoOfCopies` only accepts values `> 0`.

So if copies would become `0` or negative, it will **NOT update** the field.

Example:

- if english had 40 and you do `decreaseNoOfCopies(40)` → tries to set to `0`
- but `0 > 0` is false → so it stays `40`

If you want to allow `0`, change:

Java

to:

Java

How to print increase and decrease results separately

Option A (simplest): print after each call in `BookRunner`

```java
tamil.increaseNoOfCopies(100);
System.out.println("Tamil after increase: " + tamil.getNoOfCopies());

english.increaseNoOfCopies(80);
System.out.println("English after increase: " + english.getNoOfCopies());

tamil.decreaseNoOfCopies(0);
System.out.println("Tamil after decrease: " + tamil.getNoOfCopies());
```

Java

Option B: make increase/decrease methods print themselves

Modify `Book` like this:

```java
public void increaseNoOfCopies(int howMany) {
    setNoOfCopies(this.noOfCopies + howMany);
    System.out.println("After increasing: " + this.noOfCopies);
}

public void decreaseNoOfCopies(int howMany) {
    setNoOfCopies(this.noOfCopies - howMany);
```

Java

Option C (best style): return the updated value

```java
public int increaseNoOfCopies(int howMany) {
    setNoOfCopies(this.noOfCopies + howMany);
    return this.noOfCopies;
}

public int decreaseNoOfCopies(int howMany) {
    setNoOfCopies(this.noOfCopies - howMany);
```

Java

Then in `BookRunner`:

```java
System.out.println("Tamil after increase: " + tamil.increaseNoOfCopies(100));
System.out.println("English after increase: " + english.increaseNoOfCopies(80));
```

Java

```
System.out.println("English after decrease: " + english.decreaseNoOfCopies(50));
```

Final values in your current logic

With your current calls:

- Tamil: 100 → increase 100 → 200 → decrease 0 → **200**
- English: 10 → increase 80 → 90 → decrease 50 → **40**

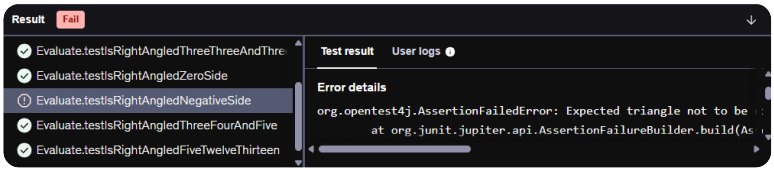
So if you print at the end:

Java

You'll see:

```
Tamil final: 200
English final: 40
```

If you tell me whether you want **0 copies to be allowed** (like out of stock), I'll adjust `setNoOfCopies` and the decrease logic the right way.




```
public class TriangleValidator {
    public boolean isRightAngled(int side1, int side2, int side3) {
        // write your code

        if(side3 <= 0 || side2 <=0 || side3 <=0 ){
```

```
            return false;
        }
        if(side1*side1 == side2*side2 + side3*side3){
            return true;
        }
        if(side2*side2 == (side1*side1) + (side3*side3)){
            return true;
        }
        if(side3*side3 == (side1*side1) + (side2*side2)){
            return true;
        }

    return false;

    }
}
```

Why am I getting error

You are very close 👍 — the failure is happening because of this line:

Java

❌ Problem 1: You forgot to check `side1`

You checked `side3` twice and **never checked** `side1`.

So when the test runs:

```
Evaluate.testIsRightAngledNegativeSide
```

It probably passes a negative value like:

Java

Since `side1` is NOT being checked, your method continues and evaluates:

```
(-3 * -3) == 4*4 + 5*5
9 == 16 + 25
9 == 41 → false
```

But in some combinations it might incorrectly pass.

The test expects:

> If any side is negative → return false immediately.

✅ Fix

Change this:

```java
```

To this:

```java
if(side1 <= 0 || side2 <= 0 || side3 <= 0){
```

⚠️ Another Important Improvement (Triangle Validity)

Right now, your method checks only Pythagoras rule.

But it does NOT check if the sides form a valid triangle.

For example:

```
1, 2, 10
```

This is NOT a triangle.

So you should also check the triangle inequality rule:

```
side1 + side2 > side3
side1 + side3 > side2
```

```
side2 + side3 > side1
```

✅ Fully Correct Version

```java
public class TriangleValidator {
    public boolean isRightAngled(int side1, int side2, int side3) {

        // Check for non-positive sides
        if(side1 <= 0 || side2 <= 0 || side3 <= 0){
            return false;
        }

        // Check triangle validity
        if(side1 + side2 <= side3 ||
           side1 + side3 <= side2 ||
           side2 + side3 <= side1){
            return false;
        }

        // Check Pythagoras
        if(side1 * side1 == side2 * side2 + side3 * side3)
            return true;

        if(side2 * side2 == side1 * side1 + side3 * side3)
            return true;

        if(side3 * side3 == side1 * side1 + side2 * side2)
            return true;

        return false;
```

Java

🎯 Why the test was failing

From your screenshot:

```
Evaluate.testIsRightAngledNegativeSide
Expected triangle not to be right angled
```

That means:

- The test passed a negative side

- Your method didn't properly reject it
- So the assertion failed

If you want, I can also show you a **cleaner mathematical way** to write this using sorting (much better approach).

```
public class PerfectNumberChecker {
    public boolean isPerfectNumber(int number) {

        if(number<=0){
            return false;
        }
        int divisorSum=0;
        for(int i=1;i<=number;i++){
            if(number%i == 0){
                divisorSum += i;
            }
        }
        return divisorSum == number;

    }
}
```

when input is 6 and 28, it should return true but showing false - what's the error here

You're very close 👍 — the mistake is small but important.

🔍 What is a Perfect Number?

A perfect number is:

> A number that is equal to the **sum of its proper divisors**
> (Proper divisors = divisors excluding the number itself)

Example:

6

Divisors:

1, 2, 3, 6

Proper divisors:

1 + 2 + 3 = **6** ✅

28

Divisors:

1, 2, 4, 7, 14, 28

Proper divisors:

1 + 2 + 4 + 7 + 14 = **28** ✅

❌ What is wrong in your code?

Your loop is:

Java

This includes the number itself.

So for 6:

1 + 2 + 3 + 6 = 12

Then you check:

Java

That becomes:

12 == 6 → false ❌

✅ Fix

You must exclude the number itself.

✔️ Corrected Code:

```java
public class PerfectNumberChecker {
    public boolean isPerfectNumber(int number) {

        if(number <= 0){
            return false;
        }

        int divisorSum = 0;

        for(int i = 1; i < number; i++){   // change <= to <
            if(number % i == 0){
                divisorSum += i;
            }
        }

        return divisorSum == number;
```

Java

🚀 Even Better (Optimized Version)

You don't need to check till `number - 1`.

You only need to check till `number/2` because:

No divisor (except the number itself) can be greater than number/2.

Java

So best version:

```java
public class PerfectNumberChecker {
    public boolean isPerfectNumber(int number) {

        if(number <= 0){
            return false;
        }
```

```java
        int divisorSum = 0;

        for(int i = 1; i <= number / 2; i++){
            if(number % i == 0){
                divisorSum += i;
            }
        }

        return divisorSum == number;
```

Java

✅ Now Results

Input	Output
6	true
28	true
10	false
-5	false

If you want, I can also show you a super-optimized √n version (used in interviews).

error: reached end of file while parsing
error explain

The error:

```
error: reached end of file while parsing
```

means:

🚨 The compiler reached the end of your file but was still expecting something (usually

a }).

🎯 Most Common Cause

✅ Missing closing curly brace }

Java uses { and } to define blocks.

If you forget one } , the compiler keeps reading until the file ends — and then throws this error.

🔎 Example of the Problem

```java
public class Test {
```

Java

❌ Missing 2 closing braces.

Compiler reaches end of file → still expecting } → error.

✅ Correct Version

```java
public class Test {
    public static void main(String[] args) {
        System.out.println("Hello");
```

Java

🔍 How to Fix in Your Code

Check:

1. Every { has a matching }
2. Your class is properly closed

3. Your method is properly closed
4. No incomplete `if`, `for`, `while`, or `class`

📌 Quick Trick

Count braces:

- Number of `{` must equal number of `}`

In most editors:

- Click next to a `{` → it highlights matching `}`

⚠️ Other Possible Causes (less common)

- Missing `)` in `if(...)`
- Missing `;`
- Unfinished string:

```
Java
```

💡 Example Related to Your Earlier Code

If you accidentally wrote:

```java
public class PerfectNumberChecker {
    public boolean isPerfectNumber(int number) {
        if(number <= 0){
```

❌ Missing:

- `}`
- `}`

Compiler will say:

```
reached end of file while parsing
```

If you paste your exact code, I can point out exactly where the missing brace is.

```
if (temperature < 0) {
        return "It's freezing!Wear a heavy coat.";
    }
    else if (temperature >= 0 && temperature <= 10){
        return "It's cold! Bundle up.";
    }
    else if (temperature >= 11 && temperature <= 20){
        return "It's Cool! A light jacket will do.";
    }
    else {
        return "It's warm! Enjoy the day.";
    }
    /*if (temperature < 0) {
        return "It's freezing! Wear a heavy coat.";
    }
    else if (temperature >= 0 && temperature <= 10) {
        return "It's cold! Bundle up.";
    }
    else if (temperature >= 11 && temperature <= 20) {
        return "It's cool! A light jacket will do.";
    }
    else {
        return "It's warm! Enjoy the day.";
    }*/
```

what's the difference in both the code which is commented

Good question 👍

Logically, **both codes work the same** — the conditions are identical.

But there are **small differences in the output strings**, and that can matter in tests.

🔎 Difference 1: Space after !

❌ First version:

Java

There is **no space** between:

```
freezing!Wear
```

✅ Commented version:

Java

There **is a space**:

```
freezing! Wear
```

👉 This difference matters if the system is checking **exact string match**.

🔎 Difference 2: Capitalization of "Cool"

❌ First version:

Java

"Cool" → capital **C**

✅ Commented version:

Java

"cool" → small **c**

Again, if this is being tested using `assertEquals()`, it must match **exactly**.

🔎 Difference 3: Minor Formatting

Spacing inside conditions is slightly different:

```
Java
```

vs

```
Java
```

But this makes **no functional difference**.

🎯 Final Answer

Functionality:

✔️ Both codes behave the same logically.

Output difference:

⚠️ The first version may fail automated tests because:

- Missing space after !
- Different capitalization ("Cool" vs "cool")

💡 Important Rule for Java Tests

In Java:

```
Java
```

String comparison is:

- Case sensitive
- Space sensitive

If your test is failing, use the commented version — it is likely the expected exact output.

If you want, I can also show you a cleaner way to write the temperature logic.

```
public class Main
{
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int num = -1;
        do{
            if(num != -1){
                System.out.println("Cube is "+ (num*num*num));
            }
            System.out.println("Enter a number ");
            num = sc.nextInt();
        }while(num >=0);
        System.out.println("Thank you!");
    }
}
```

Explain me this program, already num = -1 assigned but how input value is taken and processed

Great question 👍 This is about how a **do-while loop** works.

Let's break it step by step.

🔎 First: Important Rule of `do-while`

A **do-while loop always runs at least once**, even if the condition is false.

Structure:

```
do {
```

Java

The condition is checked **after** the loop runs once.

🧠 Now Your Code

Java

You are initializing `num` with `-1`.

But this is just a **starting value** before user input.

🚀 Step-By-Step Execution

✅ Step 1: Program starts

Java

So:

```
num = -1
```

✅ Step 2: Enter the `do` block

Because it is a `do` loop, it runs immediately.

```
if(num != -1){
```

Java

At this moment:

```
num = -1
```

Condition:

```java
```

Is:

```
-1 != -1 → false
```

So this block is skipped.

Nothing prints here.

✅ Step 3: Ask for input

```java
```

Now the user types a number.

Suppose user enters:

```
3
```

Now:

```
num = 3
```

✅ Step 4: Check while condition

```java
```

Now:

```
3 >= 0 → true
```

So loop runs again.

🔁 Second Iteration

Now:

```
num = 3
```

Step 1 inside loop:

```Java
3 != -1 → true
```

So it prints:

```
Cube is 27
```

Step 2:

It asks again:

```
Enter a number
```

User enters:

```
4
```

Now:

```
num = 4
```

Condition check:

```
4 >= 0 → true
```

Loop continues.

🔁 When does it stop?

If user enters:

```
-5
```

Then:

```
num = -5
```

Condition check:

```
-5 >= 0 → false
```

Loop stops.

Then:

```Java
```

Prints:

```
Thank you!
```

🎯 Important Concept

You asked:

> Already num = -1 assigned but how input value is taken?

Answer:

`num = -1` is just an **initial placeholder value**.

Then inside the loop this line:

```Java
```

replaces the old value with user input.

So `-1` only exists before first input.

💡 Why was `num = -1` used?

It is used so that:

```java
```

doesn't print cube before first input.

Because if num was initialized as 0:

```
Cube is 0
```

would print before user enters anything.

📃 Sample Output

If user enters:

```
3
4
-1
```

Output will be:

```
Enter a number
3
Cube is 27
Enter a number
4
Cube is 64
Enter a number
-1
Thank you!
```

🔥 Simple Summary

- `num = -1` is just a starting value

- `do` loop runs first without checking condition
- User input replaces `num`
- Loop continues while number is ≥ 0
- Negative number stops the loop

If you want, I can show you a cleaner version without using `-1` as initial value.

what is object composition in java, explain me with small example also with real life example

Great question 👍
Let's understand **Object Composition in Java** in a very simple way.

📌 What is Object Composition?

Object Composition means:

> One class contains another class as a part of it.

In simple words:

👉 A class is made up of other objects.

It represents a **HAS-A relationship**.

🏠 Real Life Example

A Car HAS-A Engine

A car is not an engine.
But a car **contains** an engine.

So:

```
Car → has an → Engine
```

This is composition.

👩‍💻 Small Java Example

Step 1: Create Engine class

```java
class Engine {
    void start() {
        System.out.println("Engine starts");
```

‹/› Java

Step 2: Create Car class that contains Engine

```java
class Car {
    private Engine engine;   // Composition

    Car() {
        engine = new Engine();   // Car creates Engine
    }

    void startCar() {
        engine.start();
        System.out.println("Car is moving");
```

‹/› Java

Step 3: Main Class

```java
public class Main {
    public static void main(String[] args) {
        Car car = new Car();
        car.startCar();
```

Java

```
}
```

🖨️ Output:

```
Engine starts
Car is moving
```

🔎 What Happened Here?

- `Car` contains an `Engine`
- When Car object is created → Engine object is also created
- Car uses Engine's method

This is object composition.

📦 Another Simple Example (Very Easy)

A Person HAS-A Heart

```java
class Heart {
    void beat() {
        System.out.println("Heart is beating");
    }
}

class Person {
    private Heart heart = new Heart();  // Composition

    void live() {
        heart.beat();
        System.out.println("Person is alive");
    }
}

public class Main {
    public static void main(String[] args) {
        Person p = new Person();
        p.live();
```

Java

```
}
```

🆚 Composition vs Inheritance

Inheritance	Composition
IS-A relationship	HAS-A relationship
Dog IS-A Animal	Car HAS-A Engine
`extends` keyword	Object inside class

🧠 Why Use Composition?

1. Reuse code
2. Keep classes modular
3. Better flexibility
4. Preferred over inheritance (in many cases)

🎯 Important Types

There are two types of composition:

1️⃣ Strong Composition

If outer object is destroyed → inner object is destroyed

Example: Person and Heart

2️⃣ Aggregation (Weak Composition)

Inner object can exist independently

Example: Student and Library

💡 Simple Definition for Exams

> Composition is a design technique where one class contains another class object to reuse its functionality, representing a HAS-A relationship.

If you want, I can also explain with a **diagram** or show how this relates to your Book example.

what is pass by reference and pass by value in java with example and real life example

This is one of the most important Java concepts 👌
Let's understand it clearly and correctly.

⚠️ First Important Truth About Java

👉 **Java is ALWAYS pass-by-value.**

But…

- For **primitive types** → value is copied.
- For **objects** → reference is copied (not the object itself).

This is why people get confused and say "pass by reference".

1️⃣ Pass By Value (Primitive Types)

📌 What it means

A copy of the variable's value is passed to the method.
Changes inside the method do NOT affect the original variable.

👩‍💻 Example

```
class Main {
    public static void main(String[] args) {
        int number = 10;
```

```
        changeValue(number);
        System.out.println(number);
    }

    static void changeValue(int num) {
```

Java

🖨️ Output:

```
10
```

🔎 Why?

- `number = 10`
- Method gets a COPY → `num = 10`
- Inside method → `num = 50`
- Original `number` is still 10

🏠 Real Life Example

Imagine:

You give your friend a **photocopy of your notes**.

If your friend writes on the photocopy,
your original notes remain unchanged.

That is pass-by-value.

2️⃣ Objects in Java (Reference is Passed by Value)

Now comes the interesting part.

When passing objects:

👉 Java passes a COPY of the reference (address).

That means:

- Both original and method reference point to SAME object.

- If object data is changed → original changes.
- If object is reassigned → original does NOT change.

👷 Example 1 (Object state changes)

```java
class Person {
    String name;
}

class Main {
    public static void main(String[] args) {
        Person p = new Person();
        p.name = "John";

        changeName(p);
        System.out.println(p.name);
    }

    static void changeName(Person person) {
        person.name = "David";
```

Java

🖨️ Output:

```
David
```

🔎 Why?

- Copy of reference is passed.
- Both references point to SAME object.
- Changing object's data affects original.

👷 Example 2 (Reassigning object)

```java
static void changePerson(Person person) {
    person = new Person();
```

Java

Now output will be:

```
John
```

🔎 Why?

Because:

- You changed the COPY of reference.
- Original reference still points to old object.

🏠 Real Life Example (Object Case)

Imagine:

You give your friend your **house key**.

- If they rearrange furniture → your house changes.
- If they throw away the key and buy a new house → your house is unaffected.

That's how object passing works.

🎯 Summary Table

Type	What is Passed	Can original change?
Primitive	Copy of value	❌ No
Object	Copy of reference	✅ Yes (if object data changes)

🧠 Final Important Statement

Java does NOT support true pass-by-reference like C++.

It is:

> Pass-by-value for primitives
> Pass-by-value of reference for objects

🔥 One Line Interview Answer

"In Java, everything is passed by value. For objects, the value passed is the reference to the object."

If you want, I can draw a memory diagram to make it 100% crystal clear.

```
class Main {
    public static void main(String[] args) {
        int number = 10;
        changeValue(number);
        System.out.println(number);
    }

    static void changeValue(int num) {
        num = 50;
    }
}
```

Explain me the working of this program step by step

Sure—here's the step-by-step working of your program.

Code

```java
class Main {
    public static void main(String[] args) {
        int number = 10;
        changeValue(number);
        System.out.println(number);
    }

    static void changeValue(int num) {
        num = 50;
```

Java

```
}
```

Step-by-step execution

1) Program starts from `main`

Java always starts execution from:

```Java
```

---

2) `number` is created and assigned `10`

```Java
```

Now in memory:

- `number` holds the value **10**

---

3) `changeValue(number)` is called

```Java
```

This means:

- The **value inside** `number` (which is `10`) is passed to the method.
- Java **copies** the value `10` into the parameter `num`.

So inside the method:

- `num` becomes **10** (a separate variable)

✅ Important: `number` and `num` are different variables.

---

4) Inside `changeValue`, `num` is changed to `50`

Java

Now:

- `num` becomes **50**
- But `number` in `main` is still **10** because only the copy was changed.

---

5) `changeValue` ends and control returns to `main`

The method finishes, and its local variable `num` is gone.

---

6) Print `number`

Java

Since `number` was never changed in `main`, it still prints:

✅ Output:

```
10
```

Why it prints 10 (simple reason)

Because `int` is a **primitive type**, and Java passes primitives **by value** (copy).

So changing `num` does **not** change `number`.

If you want, I can draw a memory diagram to make it 100% crystal clear for pass by reference

Perfect 👍 Let's draw a **memory diagram** to make pass-by-value (for primitives) and object reference passing crystal clear.

Remember:

🔴 Java is always pass-by-value
🟢 For objects, the value copied is the reference (address)

1️⃣ Primitive Example Memory Diagram

Code

```java
int number = 10;
changeValue(number);

static void changeValue(int num) {
```

Java

🧠 Step 1: Before method call

Stack Memory

```
MAIN METHOD STACK
------------------
number → 10
```

🧠 Step 2: Method is called

Java copies the value 10 into `num`.

```
MAIN STACK                changeValue STACK
-----------               -----------------
number → 10               num → 10
```

Important:

- `number` and `num` are two separate variables.

- They just both contain 10.

🧠 Step 3: Inside method

```
num = 50;
```

Now memory looks like:

```
MAIN STACK                  changeValue STACK
-----------                 -----------------
number → 10                 num → 50
```

Only `num` changed.

🧠 Step 4: Method ends

The `changeValue` stack disappears:

```
MAIN STACK
-----------
number → 10
```

Final Output:

```
10
```

2️⃣ Object Example Memory Diagram

Code

```java
class Person {
    String name;
}

Person p = new Person();
p.name = "John";

changeName(p);
```

Java

🧠 Step 1: Object Created

```
STACK (main)
-------------
p  ──────┐
         ▼
       HEAP
       ------
       Person Object
       name → "John"
```

- `p` stores the reference (address).
- Object is stored in heap.

🧠 Step 2: Method Call

Java copies the reference into `person`.

```
MAIN STACK                changeName STACK
-----------               ----------------
p  ──────┐                person ──┐
         ▼                         ▼
         HEAP                      HEAP
         ----                      ----
         Person Object
         name → "John"
```

Both `p` and `person` point to SAME object.

🧠 Step 3: Inside method

```
person.name = "David";
```

Now heap changes:

```
Person Object
name → "David"
```

Since both references point to same object:

After method ends:

```
MAIN STACK
-----------
p  ──────┐
         ▼
        HEAP
        ------
        Person Object
        name → "David"
```

Output:

```
David
```

🔥 Important Twist

If inside method you do:

```
</> Java
```

Then memory becomes:

```
MAIN STACK                changeName STACK
-----------               ----------------
p  ──────┐                person ──┐
         ▼                         ▼
        HEAP                      HEAP
        ----                      ----
        Person A                  Person B
        name → "John"             name → null
```

Now:

- `p` still points to original object.
- Only method reference changed.

After method ends → new object disappears.

So original remains unchanged.

🎯 Final Understanding

Case	What is copied?	Can original change?
Primitive	Actual value	❌ No
Object	Reference (address)	✅ Yes (if object data changes)

🧠 Super Clear One-Line Rule

Java does NOT pass objects by reference.

It passes a COPY of the reference.

If you want, I can now give you a small tricky question to test your understanding 😄

give me a small tricky question to test my understanding

Love it 😄 Let's test your understanding.

Don't run it.

Try to **predict the output first**.

🧠 Tricky Question 1

```
class Box {
    int value;
}

public class Main {
```

```java
    public static void main(String[] args) {
        Box b = new Box();
        b.value = 10;

        change(b);
        System.out.println(b.value);
    }

    static void change(Box box) {
        box.value = 20;
        box = new Box();
        box.value = 30;
```

Java

❓ Question:

What will be printed?

A) 10

B) 20

C) 30

D) Compilation Error

🧠 Tricky Question 2 (Primitive + Object)

```java
class Data {
    int num;
}

public class Main {
    public static void main(String[] args) {
        Data d = new Data();
        d.num = 5;

        modify(d, d.num);
        System.out.println(d.num);
    }

    static void modify(Data data, int x) {
        data.num = 15;
        x = 25;
```

Java

```
}
```

❓ Question:

What will be printed?

A) 5
B) 15
C) 25
D) 0

Take a minute.
Tell me your answers (Q1 and Q2), and I'll explain why 😎

30 & 15

Nice try 😄🔥
You got **one correct** and one slightly tricky.

Your answers:

- Q1 → **30** ❌
- Q2 → **15** ✅

Let's break them down clearly.

✅ Question 1

```java
static void change(Box box) {
    box.value = 20;
    box = new Box();
```

```
}
```

Step-by-step memory:

1️⃣ In main:

```
b ──────┐
        ▼
      Box Object A
      value = 10
```

2️⃣ Method call

A **copy of the reference** is passed.

```
b ──────┐
        ▼
      Box Object A
      value = 10

box ────┘
```

Both point to same object.

3️⃣ Inside method:

```java
```

Now Object A becomes:

```
Box Object A
value = 20
```

4️⃣ Then this line:

```java
```

Now:

```
b ──────┐
        ▼
       Box Object A
       value = 20

box ────┐
        ▼
       Box Object B
       value = 0
```

You created a NEW object, but only `box` points to it.

`b` still points to Object A.

5️⃣ Then:

```java
</> Java
```

This changes Object B only.

Object A is still:

```
value = 20
```

🔥 Final Output:

```
20
```

✅ Correct answer: **B) 20**

✅ Question 2 (You got this correct 👏)

```java
static void modify(Data data, int x) {
    data.num = 15;
```

Java

Step-by-step:

Before method:

```
d ──────┐
        ▼
      Data Object
      num = 5
```

Method call:

- `data` gets copy of reference
- `x` gets copy of primitive value (5)

Inside method:

```
data.num = 15;    ✅ changes object
x = 25;           ❌ only changes copy
```

After method:

```
d.num = 15
```

Final Output:

```
15
```

✅ Correct answer: **15**

🎯 Why Q1 Felt Like 30

Because many people think:

"I changed it to 30 last, so it should print 30."

But remember:

Changing the reference does NOT change the original reference in main.

🧠 Final Rule Reinforced

Java is always pass-by-value.

- Primitive → value copied
- Object → reference copied
- Reassigning reference inside method does NOT affect original

---

If you want, I can give you one **super tricky final boss level** question 😎

Yes please

Alright 😎🔥
Here comes the **Final Boss Level** question.

Take your time. Don't rush.

🧠 Final Boss Question

```
class Item {
    int value;

    Item(int value) {
        this.value = value;
    }
}

public class Main {
```

```java
    static void update(Item item1, Item item2) {
        item1.value = 100;
        item2 = item1;
        item2.value = 200;
    }

    public static void main(String[] args) {

        Item a = new Item(10);
        Item b = new Item(20);

        update(a, b);

        System.out.println(a.value + " " + b.value);
```

Java

❓ What will be printed?

A) 100 200

B) 200 200

C) 200 20

D) 100 20

🧠 Hint:

- Think carefully about:
  - When object fields are changed
  - When references are reassigned
  - Which objects `a` and `b` actually point to

Reply with your answer (A, B, C, or D), and I'll break down the memory diagram step-by-step 😄🔥

give me some more hint

Good 😎 you're thinking carefully — that's how you master this.

Let me give you **progressive hints** (don't scroll too fast, think after each one).

🔎 Hint 1 — Before Method Call

In `main`:

```java
```

Memory looks like:

```
a ──► Object A (value = 10)
b ──► Object B (value = 20)
```

Two completely separate objects.

🔎 Hint 2 — When `update(a, b)` is called

Remember:

> Java passes a COPY of the reference.

So inside `update`:

```
item1 ──► Object A
item2 ──► Object B
```

Still two different objects at this point.

🔎 Hint 3 — First Line Inside Method

```java
```

Which object changes?

Object A becomes:

```
Object A (value = 100)
```

Object B is still:

```
Object B (value = 20)
```

🔎 Hint 4 — The Critical Line

‹/› Java

⚠️ This does NOT change object B.

This changes what `item2` points to.

Now inside the method:

```
item1 ──► Object A
item2 ──► Object A
```

Object B is now unreferenced inside the method.

But remember…

`b` in main still points to Object B.

🔎 Hint 5 — Final Line Inside Method

‹/› Java

Since `item2` now points to Object A…

Which object changes?

Object A becomes:

```
Object A (value = 200)
```

Object B is still:

```
Object B (value = 20)
```

🔎 Final Question to Think About

After method ends:

- What does `a` point to?
- What does `b` point to?
- What are their values?

Now tell me:

👉 What will `System.out.println(a.value + " " + b.value);` print?

a points to object A - value is 200, b points to Object B - value is 20, so the answer is 200,20

🔥 PERFECT.

You got it exactly right.

Final Output:

```
200 20
```

Let's quickly confirm it cleanly.

✅ Final Memory State After Method Ends

In `main`:

```java
Item a = new Item(10);
```

After `update(a, b)` finishes:

ChatGPT 5.3 Instant

- Changed to 100
- Then changed to 200

  ✅ Final value = **200**

Object B

- Was 20
- Never modified directly

  ✅ Final value = **20**

References in `main`

```
a ──► Object A (value = 200)
b ──► Object B (value = 20)
```

So:

```java
```

Prints:

```
200 20
```

🎯 What This Confirms You Understand

You correctly understood:

- ✔ Objects are not passed by reference
- ✔ A copy of the reference is passed
- ✔ Reassigning a parameter does NOT change the original reference
- ✔ Changing object fields DOES affect the original

That's strong Java foundation knowledge 👏

If you want, I can now give you:

- 🧨 A question involving arrays (even trickier)
- 🧠 A recursion + reference trap
- 💼 An interview-style real-world scenario question

Your level is improving fast 😄